

RED TEAM TRADecraft · BEGINNER FRIENDLY

Introduction to Antivirus Evasion

Understanding Microsoft Defender, and slipping a payload past it.



Lachezar (@lsecqt)

Red Teaming Army · Offensive Security Researcher

DEFENDER

Who is talking at you



Lachezar

@lsecqt

- Red teamer at Accompio (r-tec IT Security GmbH)
- Offensive security researcher & payload engineering
- Founder of the Red Teaming Army community
- I teach this on YouTube, Patreon & SoftUni

What you'll leave with

- The difference between an antivirus and an EDR
- How Defender decides something is malicious
- What shellcode is, and why offensive tooling uses it
- The core evasion ideas: staging, encryption, sandbox evasion

Beginner friendly. Concepts and diagrams, no walls of code.

The plan for today

01

AV vs EDR

What each one is, and what we're up against

02

How Defender works

Signature-based and behaviour-based detection

03

Shellcode

What it is, and why we need it for evasion

04

Evasion techniques

Staging, encryption, sandbox evasion

05

Staging, visually

A diagram of the exact idea behind the demo

06

Live demo

Staging over HTTP, executed from memory

Antivirus vs EDR

Antivirus

The gatekeeper on the file

- Asks: is this file or process known to be bad?
- Leans on signatures and simple behaviour rules
- Fast, lightweight, runs on every endpoint
- Microsoft Defender lives here, and it's our target

EDR

The detective watching everything

- Endpoint Detection & Response
- Records rich telemetry across the whole system
- Correlates events over time to spot attack chains
- Heavier, and a much harder target, out of scope today

SCOPE

Today we fight the antivirus

An EDR is a different, bigger conversation. To learn the fundamentals, we start where everyone starts.



In scope

- Microsoft Defender
- Signature-based detection
- Behaviour-based detection
- Getting shellcode to run in memory



Out of scope

- Full EDR platforms
- Kernel-level telemetry
- Threat hunting & response
- Anything you're not authorised to test

SECTION 01

How Defender works

Two questions the antivirus asks about your code.

Signature-based detection

The idea

"Have I seen these exact bytes before?"

- Defender keeps a huge database of known-bad patterns
- It scans your file and looks for a match
- A match means blocked before the code ever runs
- Cheap and fast, so it is the very first gate

Strong against

Known tools and payloads. Anything already in the signature database gets caught instantly.

Its blind spot

It only knows what it has already seen. Change the bytes enough and there is no signature to match.

Behaviour-based detection

The idea

"Forget the bytes, what is it doing?"

- Defender also watches what a program does as it runs
- Suspicious actions raise a score, not just suspicious files
- Touching memory in odd ways is a classic trigger
- This is why 'just rename it' stopped working years ago

Strong against

Payloads that look clean at rest but behave badly the moment they execute.

Its blind spot

It has to judge normal from abnormal. Activity that blends in with real software is hard to flag.

SO WHERE DOES THAT LEAVE US

Two gates, one goal

1

Beat the signature

Make sure nothing on disk matches a known pattern.

2

Beat the behaviour

Run in a way that does not look obviously malicious.

The rest of this talk is about doing both at once, and it starts with one key building block: shellcode.

SECTION 02

Shellcode

The building block that makes custom execution possible.

What is shellcode?

In one sentence

A small, self-contained block of machine code that can run on its own, without being a normal program file.

- It is just raw bytes, not an .exe on disk
- Position independent: it runs wherever you place it
- Small enough to move around and hand to a loader
- The standard currency of offensive payloads

Think of it as

A payload, detached from any file, that you can carry, hide, and then run on demand.



Because it is detached from a file, we control exactly how and where it runs.

Why do we need shellcode?

1

Runs from memory

It never has to sit on disk as a file, so the signature scanner has far less to grab onto.

2

Custom execution

We decide how it gets loaded and launched. That freedom is the whole game in evasion.

3

Wraps our techniques

Staging, encryption, sandbox checks: all of them are built around a block of shellcode.

SECTION 03

Evasion techniques

The building block is shellcode. These are the ways we deliver it quietly.

Three ideas we'll cover

1

Staging

Keep the payload off the machine until the last moment, then pull it in.

BEATS THE FILE SCAN

2

Encryption

Store the payload scrambled so a scanner sees noise, not a signature.

BEATS THE SIGNATURE

3

Sandbox evasion

Notice when you're being watched, and stay quiet until it's safe.

BEATS THE BEHAVIOUR CHECK

Staging

The idea

Don't ship the payload with your program. Fetch it only when you're ready to run.

- The delivered file is small and looks harmless
- The real payload lives elsewhere, on your server
- It arrives at runtime, straight into memory
- Nothing malicious ever sits on disk to be scanned

What it defeats

The file scan. There is simply nothing incriminating in the file Defender inspects, because the payload isn't in it yet.

Encryption

The idea

Store the payload scrambled. A scanner reads gibberish, and only your loader can unlock it.

- The payload bytes are encrypted, not readable
- No recognisable pattern for a signature to match
- Your loader decrypts it only in memory, at the last moment
- Often paired with staging for two layers of cover

What it defeats

Signature-based detection. There is no known pattern to match when the bytes are scrambled.

Sandbox evasion

The idea

Defender may run your code in a cage first. If you can tell, you simply behave until you're out.

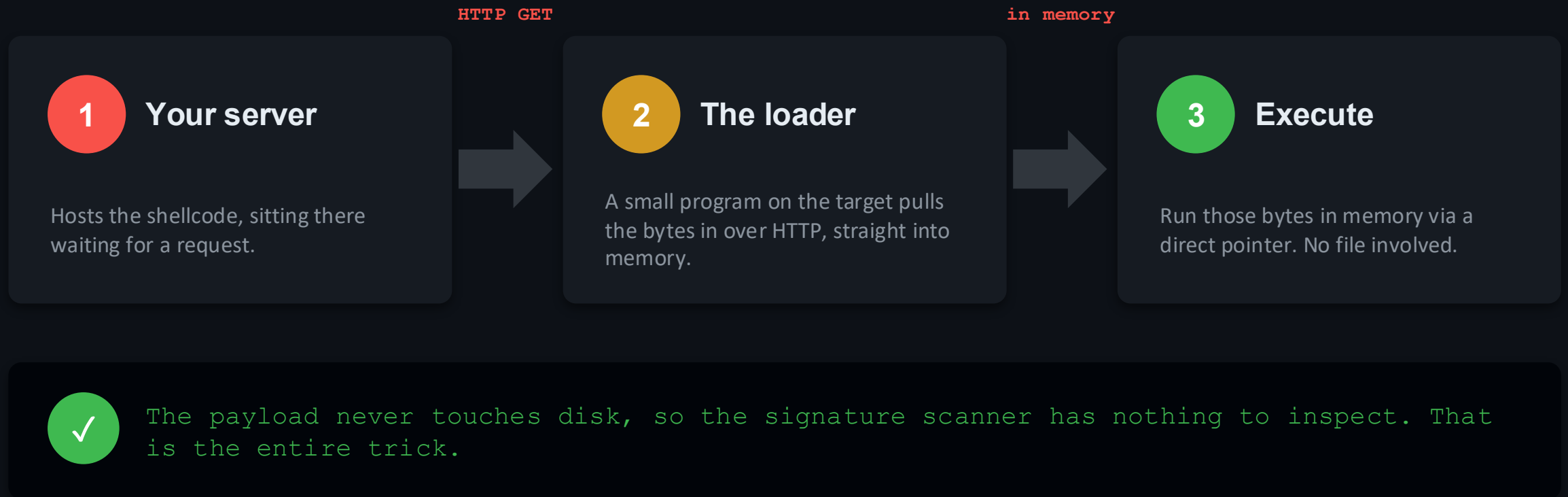
- Antivirus can detonate suspicious code in a sandbox
- Sandboxes look subtly different from a real machine
- Check the surroundings before doing anything of note
- Look boring while watched, act only when it's real

What it defeats

Behaviour-based detection. If the payload does nothing interesting while observed, there is nothing to score against it.

HOW STAGING WORKS

The idea behind the demo



LIVE

Demo time

Staging over HTTP, then executing from memory via a direct pointer, against Defender.

- 1 Host the shellcode on a simple HTTP server I control
- 2 Loader pulls it into memory at runtime, nothing lands on disk
- 3 Execute via a direct pointer, and watch Defender stay quiet



Thanks for listening

Questions now, or find me online.

X / TWITTER

@lsecqt

COMMUNITY

Red Teaming Army

YOUTUBE & PATREON

lsecqt